

BrokerVerse

Out-of-the-box insurance broking platform

Solution Architecture · Document 02 of 11

Database Design

Product	BrokerVerse
Document	02 Database Design
Version	1.0
Date	29 September 2026
Prepared by	iorta TechNXT
Classification	Confidential: for the client, iorta TechNXT and authorised partners

Document control

Document	BrokerVerse Solution Architecture: 02 Database Design
Document ID	BV-SA-02
Version	1.0
Status	Issued for review
Date	29 September 2026
Prepared by	iorta TechNXT
Source baseline	BrokerVerse source repository, branch brokerverse-platform (backend/, brokerverse/, docker-compose.yml, render.yaml, docs/)

Version history

Version	Date	Author	Change
1.0	29 September 2026	iorta TechNXT	Initial issue

Review and approval

Role	Name	Signature	Date
Solution architect			
DevOps / infrastructure lead			
Information security			
Business owner			

Conventions

- Statements of fact are derived from the code, configuration and database of the baseline above (file paths are given relative to the repository). Figures measured on the local test system are labelled **test data** or **indicative**.
- Targets, sizes, retention periods and thresholds that are not facts of the code are labelled **Recommended / to be confirmed by the business and DevOps**.
- Items marked **Gap** are defects or missing capabilities found while preparing this document.

Contents

1 Introduction.....	4
1.1 Purpose and scope.....	4
1.2 Database platform.....	4
2 Schema overview by domain.....	5
3 Design conventions.....	6
3.1 Keys.....	6
3.2 Column types.....	6
3.3 JSONB document columns.....	6
3.4 Document numbering.....	7
3.5 Audit, history and soft delete.....	8
3.6 Constraints and indexes.....	8
4 Entity-relationship diagrams.....	10
4.1 Sales and policy.....	11
4.2 Renewals.....	12
4.3 Billing, receipts and ledger.....	14
4.4 Disbursements and petty cash.....	14
4.5 Commission and incentive.....	16
4.6 Claims.....	18
4.7 Remittance and direct bill.....	18
4.8 Reinsurance.....	20
4.9 Configuration, security and platform.....	20
4.10 Reference masters and product configurator.....	23
5 Migrations.....	25
6 Seed data.....	26
7 Time zone, date and amount handling.....	27
8 Related documents.....	28
9 Glossary.....	29

1 Introduction

1.1 Purpose and scope

This document describes the logical and physical design of the BrokerVerse database: the schema organised by business domain, the entity-relationship structure of each domain, the key tables and their keys, the use of JSONB document columns, document numbering, audit and soft-delete conventions, the migration and seed approach, and the handling of time zones, dates and amounts. The table-by-table inventory with sizes, row counts and retention classes is document 03.

Sources: `backend/src/db/migrations/*.sql` (32 files, 1,678 lines), `backend/src/db/seeds`, `backend/src/db/pool.js`, `migrate.js`, `seed.js`, `backend/src/lib/dates.js`, `money.js`, and the catalogue of the local PostgreSQL 16 database `brokerverse` (snapshot in `docs/architecture/tools/data/db_snapshot.json`).

1.2 Database platform

Item	Value (local test database)
PostgreSQL version	16.13 (Ubuntu 16.13-0ubuntu0.24.04.1)
Database size	20.2 MB (test data)
Server time zone	Etc/UTC
Tables (schema public)	102
Columns by type	text 952, timestamp with time zone 233, numeric 178, integer 89, jsonb 77, date 64, bigint 30, boolean 21, ARRAY 2
Indexes	226 (161 unique incl. 102 primary keys, 15 partial, 1 GIN)
Foreign keys	174 (128 without a supporting index)
Identity sequences (serial / identity columns)	46
Document-number counters (table sequences)	32 counters for 2026
Extensions	pgcrypto 1.3, plpgsql 1.0
Application functions	gl_accounts_defaults, jv_check_posting, next_number, numbering_prefix, rpt_age_bucket, set_updated_at
Triggers	gl_accounts_defaults_trg (gl_accounts), jv_check_posting_trg (journal_vouchers), roles_updated (roles), users_updated (users)
Views / materialised views	none
Migrations applied	32 (0001_core.sql to 0110_brokerverse_ootb_branding.sql)
Application settings (app_settings)	273 keys in 31 groups
Scheduled jobs (scheduled_jobs)	9
Snapshot taken	2026-09-29T09:25:28+00:00

One database and one schema (`public`) hold all data. The application connects with a single login through `DATABASE_URL` (node-postgres pool, maximum 10 connections per API process). The login needs rights to create tables because the API applies its own migrations on start. Only the `pgcrypto` extension is used (for `gen_random_bytes()` in primary-key defaults).

2 Schema overview by domain

The 102 tables fall into 15 domains. Row counts and sizes are from the local test database and are **test data**.

Domain	Tables	Rows (test data)	Size KB (test data)	Tables
Security and identity	9	1,047	712	login_history, password_history, password_resets, permissions, refresh_tokens, role_permissions, roles, user_roles, users
Platform services	13	2,327	1,608	agent_events, app_settings, audit_log, documents, email_outbox, job_queue, job_runs, master_records, master_types, notifications, scheduled_jobs, schema_migrations, sequences
Reference masters	16	232	704	banks, branches, cities, countries, coverages, currencies, districts, insurance_companies, policy_types, postal_codes, products, signatories, states, vehicle_brands, vehicle_models, vehicle_variants
Product configurator	4	73	280	product_components, product_risk_mappings, product_risk_sections, product_templates
Sales and policy	6	103	688	clients, endorsements, leads, policies, policy_payments, quotes
Claims	3	134	280	claim_field_changes, claim_history, claims
Renewals	7	57	416	renewal_activities, renewal_batch_policies, renewal_batches, renewal_notices, renewal_quotes, renewals, winback_campaigns
Billing, receipts and collections	6	156	432	collection_actions, collection_items, receipt_applications, receipt_lines, receipts, receivables
General ledger	5	531	520	accounting_periods, entry_matches, gl_accounts, journal_lines, journal_vouchers
Disbursements and petty cash	9	41	448	checkbooks, disbursements, invoice_lists, petty_cash_disbursements, petty_cash_funds, petty_cash_receipts, petty_cash_replenishments, petty_cash_request_lines, petty_cash_requests
Commission	2	63	152	commission_referrers, commissions
Remittance and direct bill	9	105	568	commission_debit_note_collections, commission_debit_note_lines, commission_debit_notes, direct_bill_items, remittance_approvals, remittance_delegations, remittance_items, remittance_lines, remittances
Reinsurance	7	37	320	cessions, reinsurance_bordereaux, reinsurance_exceptions, reinsurance_reconciliations, reinsurance_recoveries, reinsurance_treaties, reinsurers
Incentive	3	26	144	incentive_calculations, incentive_programs, incentive_results
Reporting	3	134	272	generated_reports, report_definitions, report_schedules
Total	102	5,066	7,544	

3 Design conventions

3.1 Keys

Convention	Detail
Business records	43 tables use a text primary key made of a readable prefix and 16 (or 12) random hexadecimal characters, generated by the database: for example <code>policies.id</code> <code>DEFAULT 'pol_' encode(gen_random_bytes(8), 'hex')</code> . Prefixes include <code>ld_</code> lead, <code>cl_</code> client, <code>qt_</code> quotation, <code>pol_</code> policy, <code>end_</code> endorsement, <code>clm_</code> claim, <code>rnw_</code> renewal, <code>or_</code> receipt, <code>jv_</code> journal voucher, <code>pv_</code> payment voucher, <code>rm_</code> remittance, <code>dn_</code> debit note, <code>usr_</code> user, <code>doc_</code> document. The ids are not guessable and never collide across tables.
Reference and log tables	46 tables use integer or bigint serial keys (masters such as countries, banks, insurers, products; and high-volume children and logs such as <code>journal_lines</code> , <code>audit_log</code> , <code>login_history</code> , <code>job_runs</code> , <code>claim_history</code>).
Natural and composite keys	<code>app_settings(key)</code> , <code>gl_accounts(code)</code> , <code>master_types(code)</code> , <code>report_definitions(code)</code> , <code>accounting_periods(period)</code> , <code>incentive_calculations(batch_id)</code> , <code>refresh_tokens(jti)</code> , <code>schema_migrations(name)</code> , <code>sequences(name, period)</code> , <code>role_permissions(role_id, permission_id)</code> , <code>user_roles(user_id, role_id)</code> .
Business numbers	Human-readable document numbers (<code>QT-2026-00001</code> , <code>POL-2026-00001</code> , <code>OR-2026-00019</code>) are separate unique columns (<code>quote_number</code> , <code>policy_number</code> , <code>receipt_number</code> , <code>jv_number</code> ...), never primary keys; the API accepts either the id or the number on most detail endpoints.
Foreign keys	174 foreign keys: 144 <code>NO ACTION</code> (a referenced record cannot be deleted while used), 26 <code>CASCADE</code> (children that have no meaning without their parent: journal lines, receipt lines, claim history, renewal notices, role grants, refresh tokens, notifications ...) and 4 <code>SET NULL</code> .

3.2 Column types

Data	Type and handling
Amounts	<code>numeric</code> (178 columns), typically <code>numeric(14,2)</code> or unconstrained; converted to JavaScript numbers by the pool type parser (<code>pg.types.setTypeParser(1700, Number)</code>); all rounding through <code>round2()</code> in <code>lib/money.js</code> (half away from zero, same rule as PostgreSQL <code>round(numeric, 2)</code>).
Business dates	<code>date</code> (64 columns: inception, expiry, due, loss, receipt and voucher dates), returned to the application as <code>YYYY-MM-DD</code> strings (type parser 1082) so no time-zone shift can occur.
Instants	<code>timestamptz</code> (233 columns, e.g. <code>created_at</code> , <code>approved_at</code> , <code>sent_at</code>); no <code>timestamp without time zone</code> columns exist.
Flexible parts	<code>jsonb</code> (77 columns in 48 tables), see below.
Codes and statuses	<code>text</code> with application-level validation (zod, service rules) and 25 <code>CHECK</code> constraints; statuses are lower-case codes mapped to display labels through settings (for example <code>claims.status_labels</code>).

3.3 JSONB document columns

JSONB is used where the structure varies by product, screen or configuration, while the columns that are searched, joined, totalled or reported on stay relational.

Table	JSONB columns and use
quotes	<code>vehicle</code> , <code>coverage</code> (wizard inputs and selected covers), <code>doc</code> (full quotation document including premium breakdown and customer data)

Table	JSONB columns and use
policies	details (risk and party details, renewal link), doc (policy document data carried from the quotation)
claims	details, driver, third_party, policy_info, adjuster, settlement
endorsements	changes (field changes and re-pricing), endorsement_type_ids, completion
renewals, renewal_quotes	coverage details, accessories, order summary, limits, premium breakdown; taxes and rating
product_templates, product_components	config, features, insurers, tags; component data (covers, rating tables, rules, the motor CTPL tariff)
master_records	data: all fields of generic masters (GIN index master_records_data_idx, jsonb_path_ops)
master_types	fields, unique_keys: definition of each master screen
app_settings	value: every configuration value (strings, numbers, arrays, objects)
audit_log	before_data, after_data of every audited change
reference masters	attrs on banks, branches, countries, states, cities, currencies, coverages, insurers, products, policy types, signatories, vehicle masters: extra attributes without schema change
finance and reinsurance	remittances.data, remittance_items.data, remittance_approvals.history, reinsurance_treaties.terms, reinsurance_recoveries.documents, incentive_programs.structure / eligibility
operations	scheduled_jobs.params, job_runs.output, job_queue.payload, generated_reports.params / totals, report_definitions.parameters / default_columns

3.4 Document numbering

Document numbers are allocated by the SQL function `next_number(name, prefix, width = 5)`: it upserts the row `(name, year)` in the table `sequences`, increments `value` atomically (`INSERT ... ON CONFLICT DO UPDATE ... RETURNING`) and returns `<prefix>-<YYYY>-<value padded to 5>`. The counter restarts every calendar year; the prefix comes from the setting `numbering.<entity>.prefix` (43 numbering settings; `numbering_prefix()` reads them in SQL). When the number is taken inside the saving transaction (for example policy issue, quotations, receipts, disbursements), a rolled-back save also rolls back its number; helpers that take the number outside a transaction (claims, renewals, masters) can leave gaps after a failed save, and additionally re-check uniqueness against the target table. If gap-free numbering is a regulatory requirement for a document type (for example official receipts), this should be confirmed per document type.

Counter (sequences.name)	Year	Last value (test data)	Counter	Year	Last value (test data)
adjustment	2026	4	petty-cash-request	2026	2
bank_txn	2026	5	policy	2026	3
bordereau	2026	2	quote	2026	11
cession	2026	13	receipt	2026	23
claim	2026	2	receipt-txn	2026	28
client	2026	2	remittance	2026	18
commission_debit_note	2026	1	remittance_batch	2026	3
disbursement-txn	2026	23	remittance_bill	2026	4
dn_collection	2026	2	remittance_exception	2026	4
endorsement	2026	3	remittance_notice	2026	3
incentive_calc	2026	4	renewal	2026	2
invoice	2026	4	ri_reconciliation	2026	2

Counter (sequences.name)	Year	Last value (test data)	Counter	Year	Last value (test data)
invoice-list	2026	6	ri_recovery	2026	6
journal	2026	117	settlement	2026	2
lead	2026	2	transfer	2026	4
petty-cash	2026	3	voucher	2026	23

Gap: `next_number()` takes the year from `now()` in the database session time zone (UTC on the local server and by default on RDS), not from the business time zone `general.timezone` (Asia/Manila). Documents created between 00:00 and 08:00 Manila time on 1 January are numbered in the previous year. Fix: compute the year with `now() AT TIME ZONE 'Asia/Manila'` (or the setting) or set the database / role time zone to Asia/Manila (recommended).

Note: The numbering helper exists in four variants in the code (`documents/common.js`, `claims/util.js`, `masters/helpers.js`, `disbursements/service.js` `nextNo`), all calling the same SQL function; consolidating them in `lib/` is a code-quality improvement, not a correctness issue.

3.5 Audit, history and soft delete

Convention	Detail
Row stamps	<code>created_at</code> (79 tables), <code>updated_at</code> (62), <code>created_by</code> (59), <code>updated_by</code> (39). <code>updated_at</code> is set by the application; database triggers maintain it only on users and roles (<code>set_updated_at()</code>).
Audit log	<code>audit_log(id, at, user_id, username, entity, entity_id, action, before_data, after_data, ip)</code> , written by <code>lib/audit.js</code> from the route handlers of every create, update, approval, posting, run and sign-in related event; indexed on (<code>entity, entity_id</code>).
Domain histories	<code>claim_history</code> (status), <code>claim_field_changes</code> (field-level), <code>renewal_activities</code> , <code>renewal_notices</code> , <code>collection_actions</code> , <code>remittance_approvals.history</code> , <code>login_history</code> , <code>password_history</code> , <code>job_runs</code> .
Soft delete	<code>leads</code> and <code>quotes</code> carry <code>deleted_at</code> ; masters and product components use a <code>status</code> of <code>deleted</code> / <code>Deleted</code> (unique indexes are partial: <code>WHERE status <> 'deleted'</code>); financial documents are never deleted but cancelled or reversed (<code>receipts.cancelled_at</code> , journal reversal and correction vouchers <code>reversal_of</code> , <code>reversed_by_jv</code> , <code>correction_of</code>). A draft quotation is the only business record deleted physically.
Financial integrity	Trigger <code>jv_check_posting</code> refuses to post a journal voucher that has fewer than two lines, is unbalanced or has a zero total, or whose accounting period (<code>accounting_periods</code>) is closed. Trigger <code>gl_accounts_defaults</code> derives the normal balance of an account from its type.

3.6 Constraints and indexes

The schema has 226 indexes: 102 primary keys, 59 further unique indexes (business numbers, natural keys, partial uniqueness such as one open renewal per policy `renewals_open_policy_uq` or one pending approval per entity `remittance_approvals_open_uq`), 15 partial indexes and one GIN index. Case-insensitive uniqueness uses expression indexes on `lower(...)` (reinsurer names, treaty numbers, programme codes, template codes).

Gap: 128 of the 174 foreign keys have no index whose leading column is the foreign-key column (list in document 03). Most are low-volume references to masters, but several are on growing tables and are used in joins or in lookups by parent: for example `job_runs.job_id` (queried by the scheduler on every run and by the run-history

screen), quotes.client_id, receipts.client_id, receipt_applications.receipt_id, remittance_lines.remittance_id, commissions.disbursement_id, incentive_results.calculation_id. They also slow down deletes of parent rows. Recommended: add indexes for the FKs of transactional and log tables before the data volume grows (to be confirmed with a query-plan review in UAT).

4 Entity-relationship diagrams

The diagrams are generated from the database catalogue. Each box lists the primary key (PK), up to two unique keys (UK), the foreign keys (FK; a red asterisk marks an FK without a supporting index) and the JSONB columns ({ }). Dashed boxes are tables of another domain. References to [users](#) (creator, owner, approver) are listed as columns but not drawn, except in the security diagram.

4.1 Sales and policy

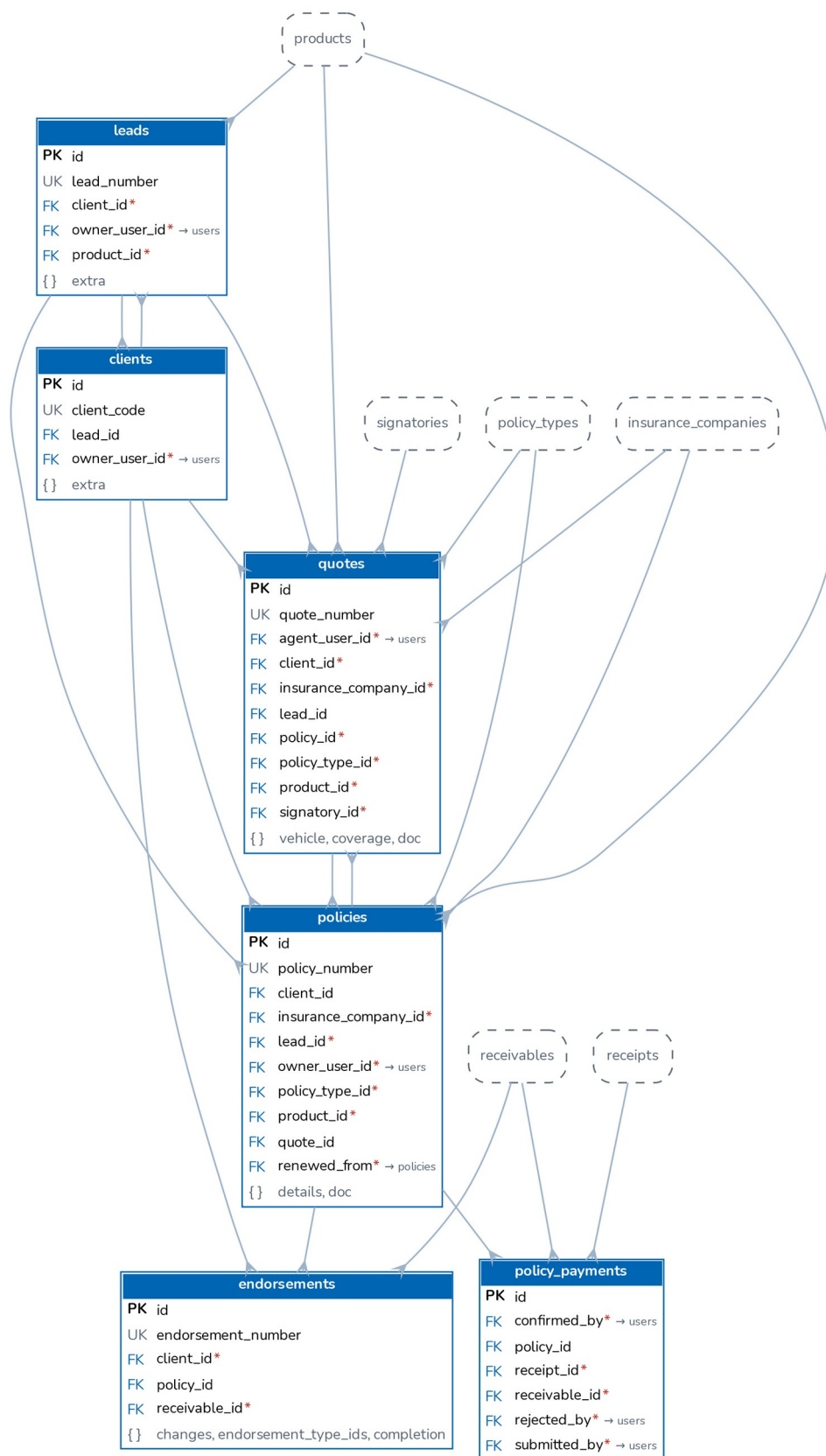


Figure 1. Sales and policy: leads, clients, quotations, policies, endorsements, payment captures

A lead becomes a client when a quotation is converted (`clients.lead_id` is unique when set). A quotation references the lead or client, product, policy type, insurer and signatory; conversion creates the policy

(`policies.quote_id`, `quotes.policy_id`). A renewed policy points to its predecessor (`policies.renewed_from`, `renewed_to`). Endorsements belong to a policy and may raise their own bill (`endorsements.receivable_id`). `policy_payments` holds payments captured by front-office staff until finance confirms them and issues a receipt.

4.2 Renewals

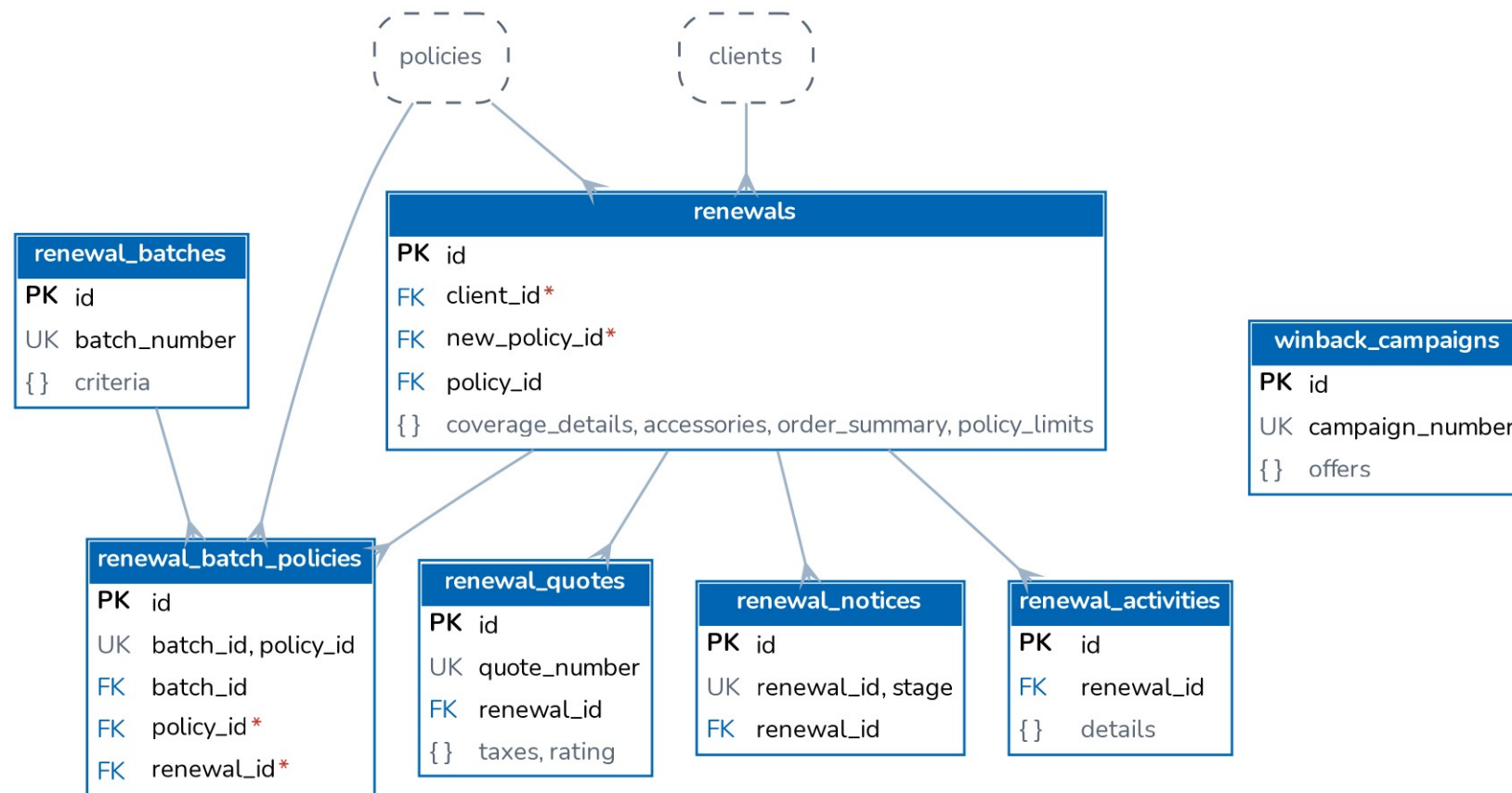


Figure 2. Renewals: renewal cases, quotes, notices, activities, batches, win-back campaigns

4.3 Billing, receipts and ledger

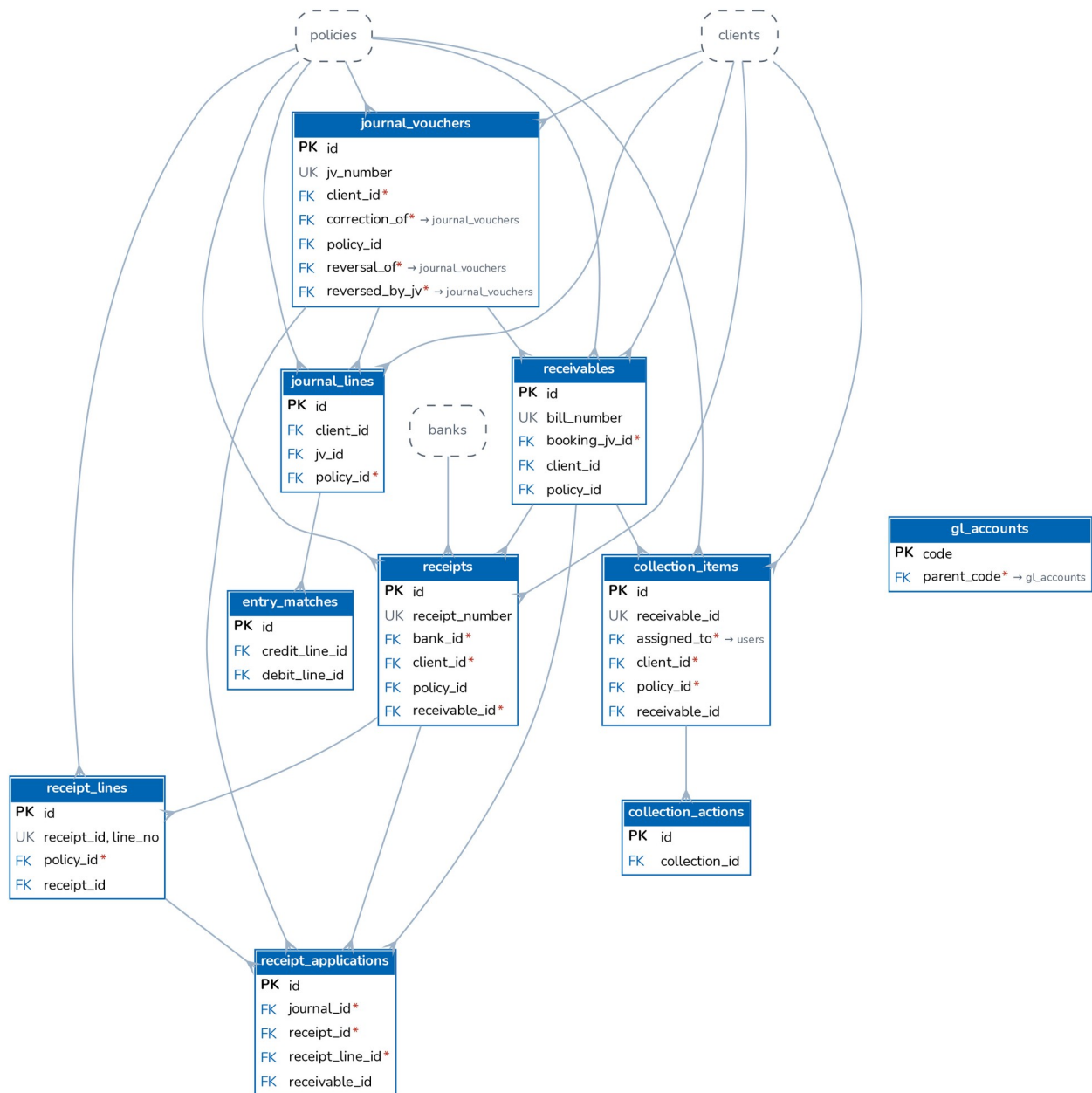


Figure 3. Billing, receipts, collections and general ledger

A policy, endorsement or renewal raises a receivable (bill) with its booking journal (`receivables.booking_jv_id`). A receipt has lines per policy, and receipt applications allocate the lines to bills and link the posting journal. Collection items track follow-up of open bills. Journal vouchers hold balanced journal lines coded to the chart of accounts (`gl_accounts`); open-item accounts are reconciled by `entry_matches` (debit line to credit line).

4.4 Disbursements and petty cash

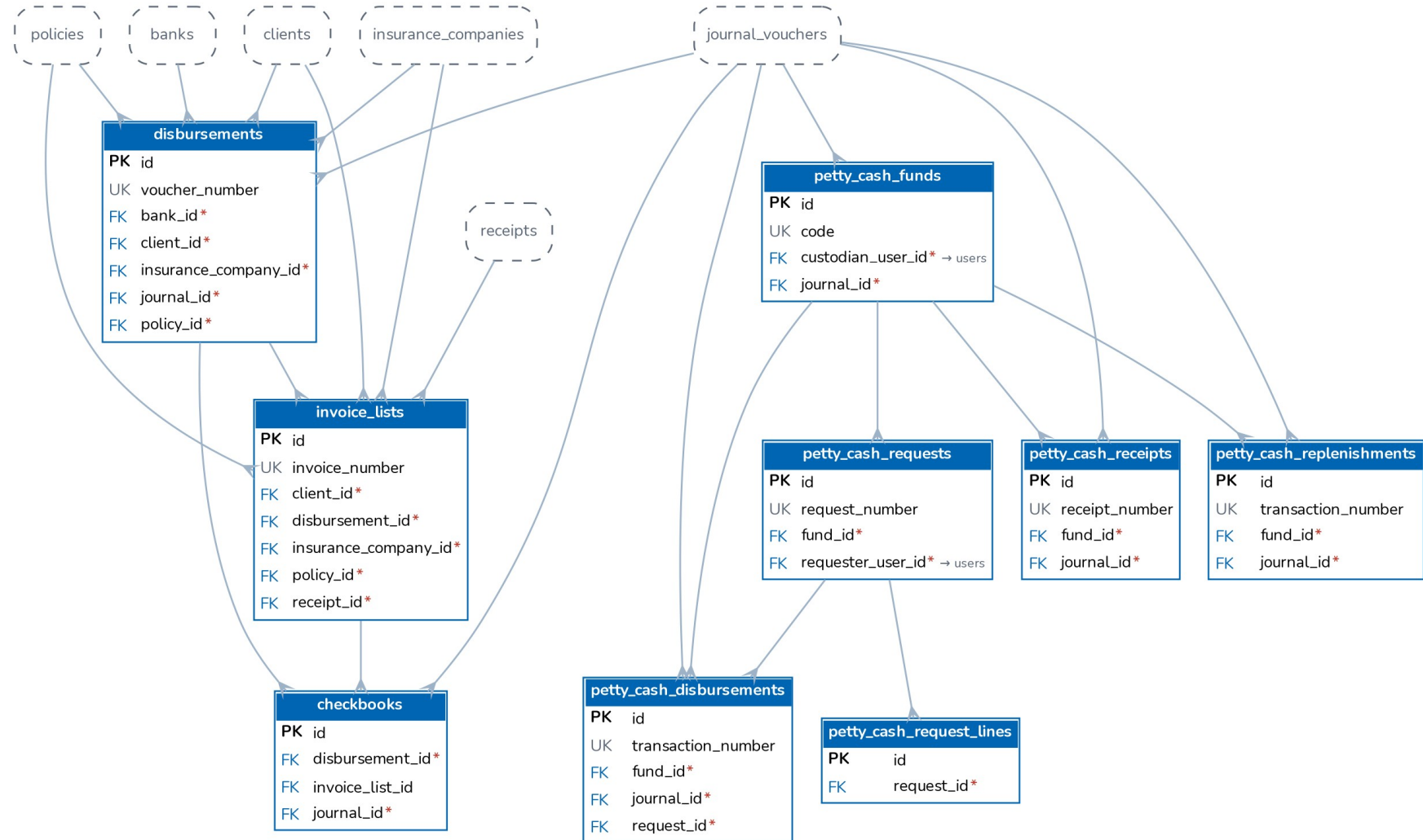


Figure 4. Disbursements (payment vouchers, invoice lists, cheques) and petty cash

4.5 Commission and incentive

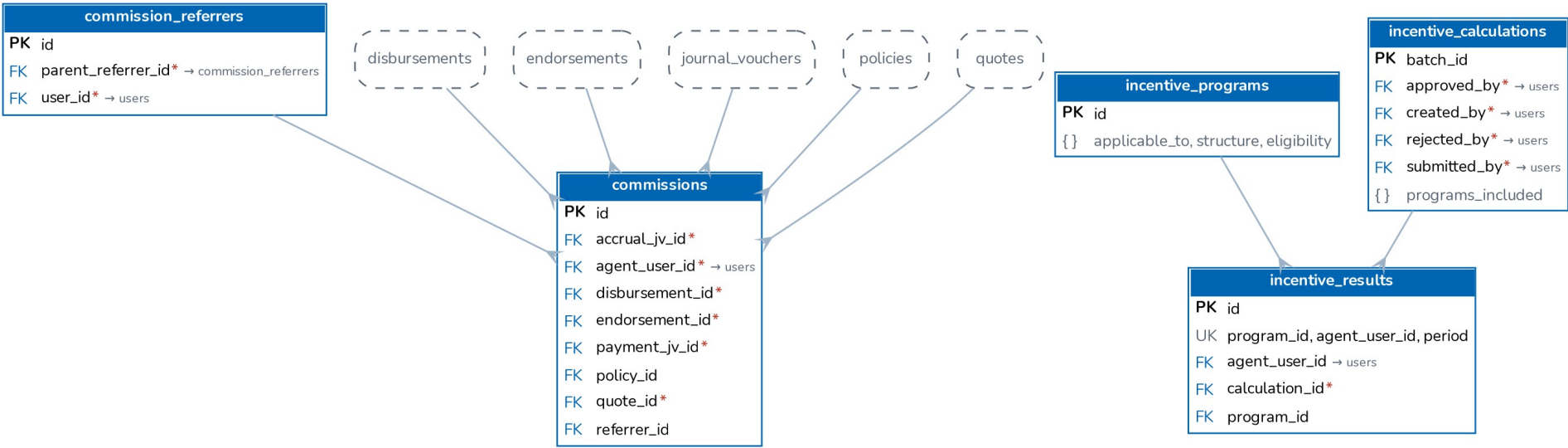


Figure 5. Commission lines, referrers and incentive programmes

Commission lines are created per policy, endorsement and referrer in the referral chain (commissions_policy_referrer_uq on policy, referrer and chain position), link to the accrual and payment journals and to the payment voucher that paid them. Referrers form a hierarchy (parent_referrer_id) and may be linked to a user.

4.6 Claims

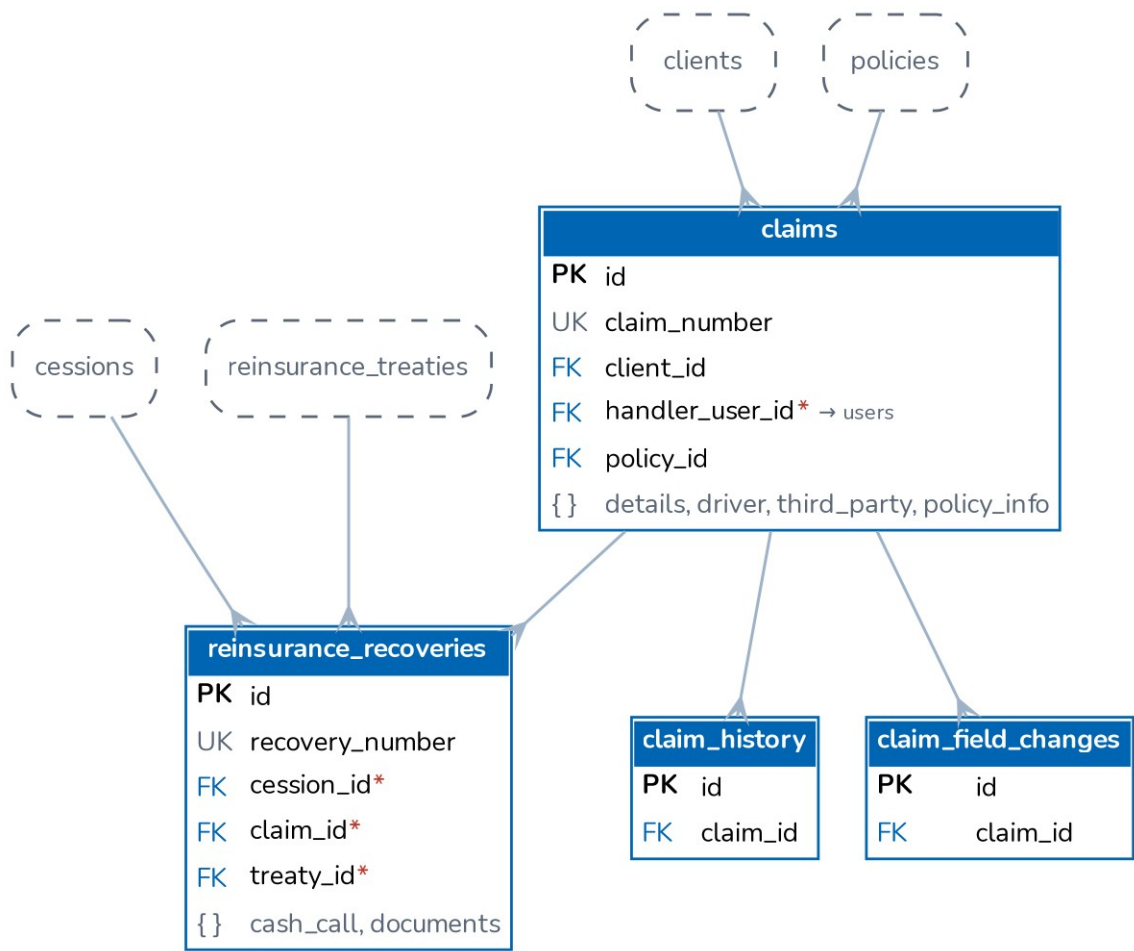


Figure 6. Claims, claim history, field-level changes and reinsurance recoveries

4.7 Remittance and direct bill

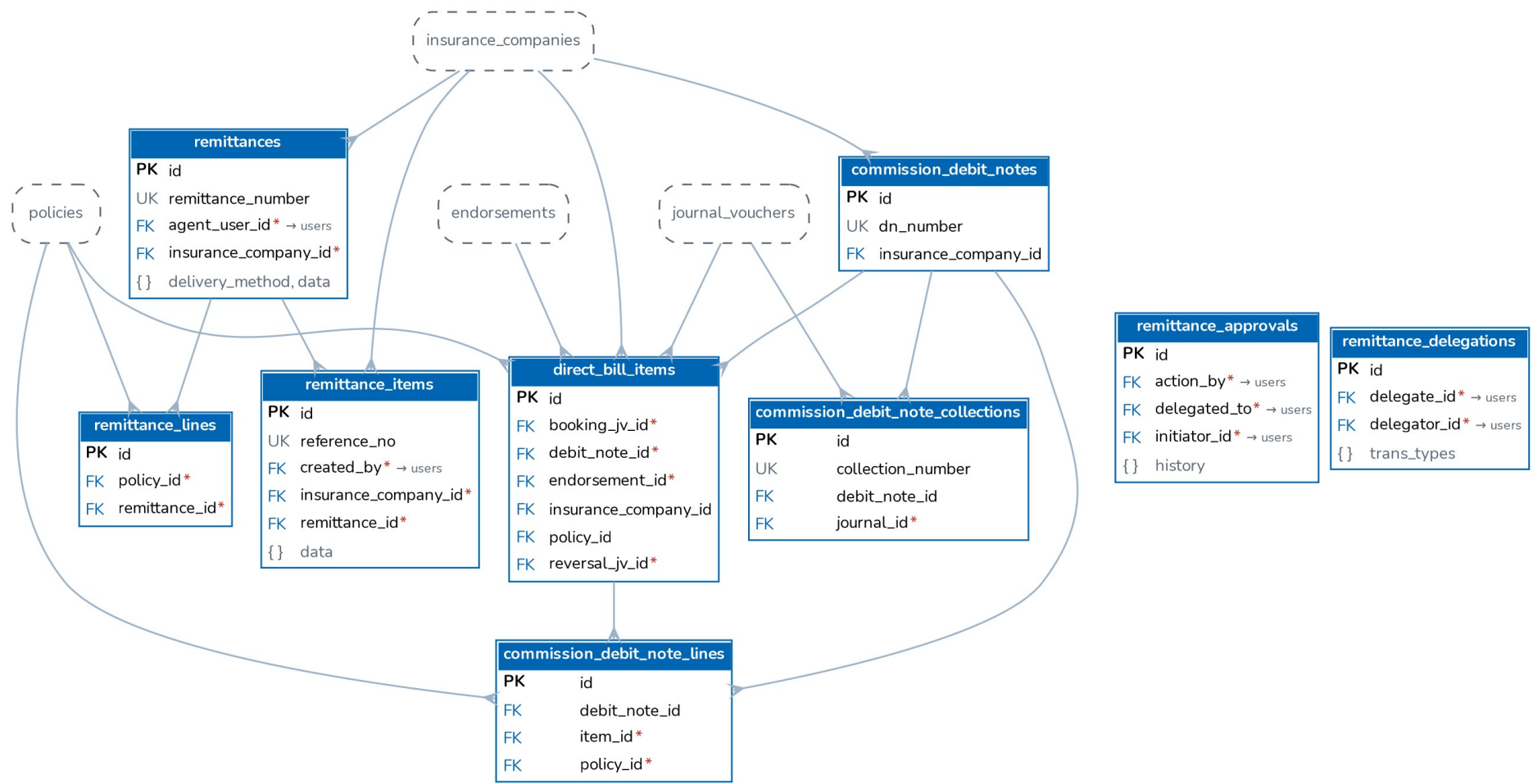


Figure 7. Remittances to insurers and direct-bill commission debit notes

Broker-billed premium is remitted to insurers through `remittances` with `remittance_lines` per policy; settlements, adjustments and exceptions are `remittance_items`; `remittance_approvals` is a generic multi-level approval register (entity, entity id, levels, SLA, history) also used by other finance approvals. Direct-billed policies produce `direct_bill_items` (commission receivable) that are billed to the insurer on `commission_debit_notes` and settled by `commission_debit_note_collections`.

4.8 Reinsurance

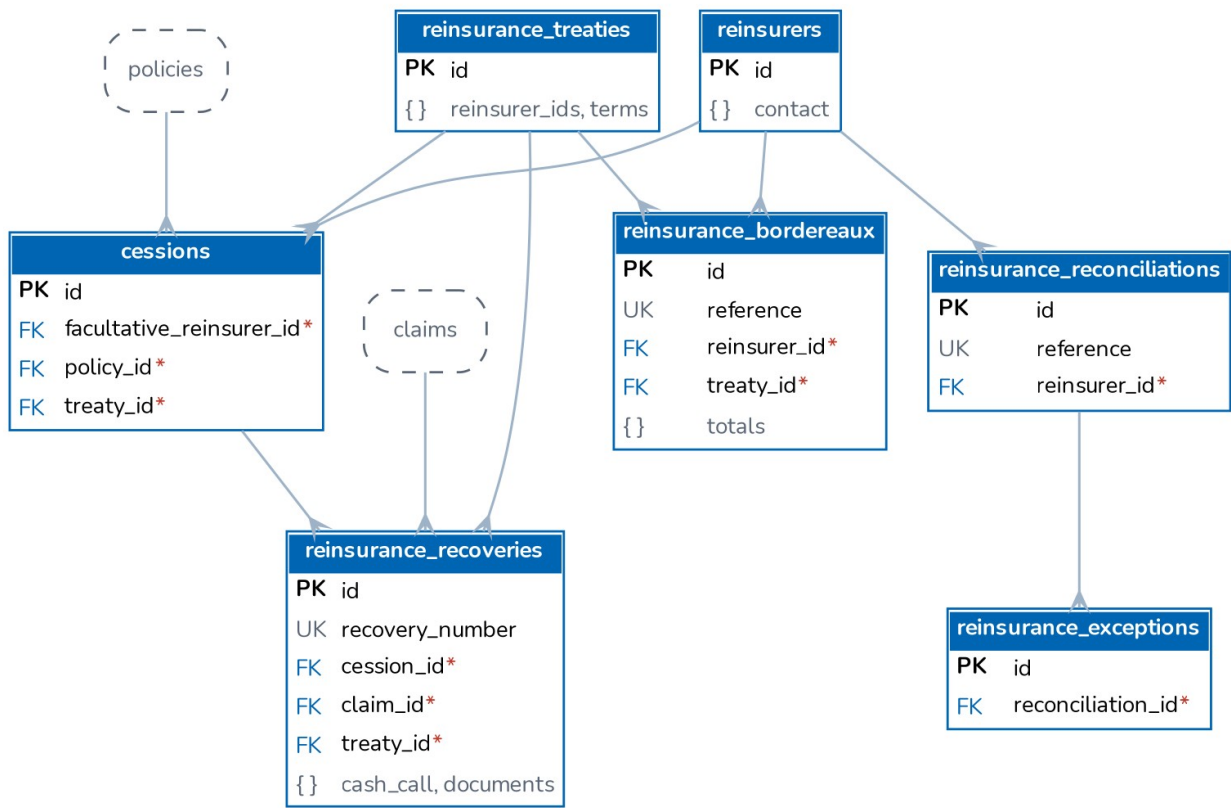


Figure 8. Reinsurers, treaties, sessions, recoveries, bordereaux and reconciliations

4.9 Configuration, security and platform

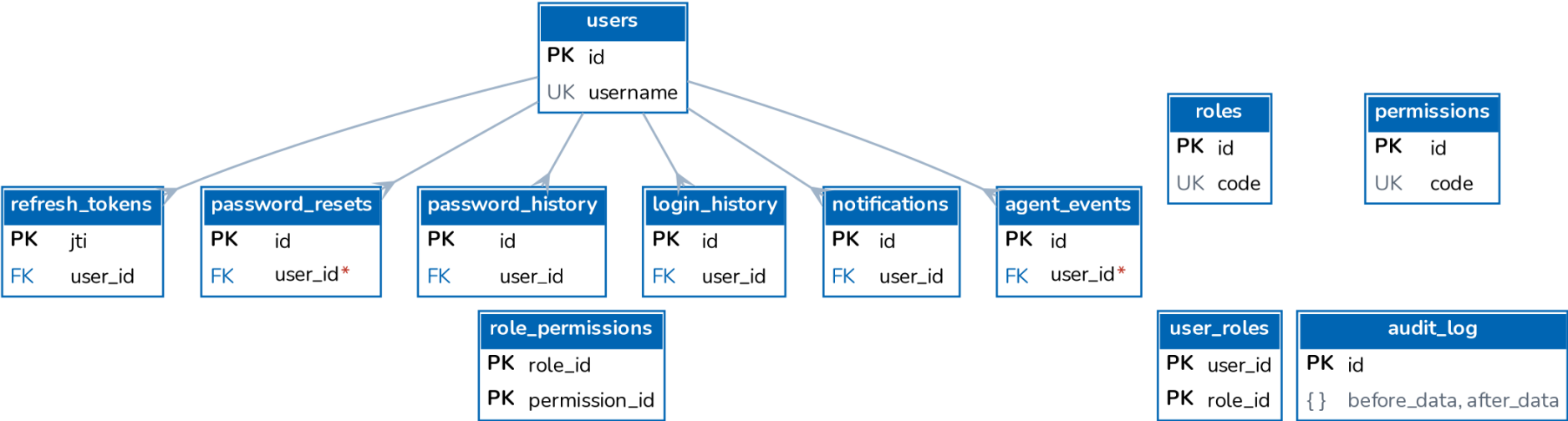


Figure 9. Security and identity tables

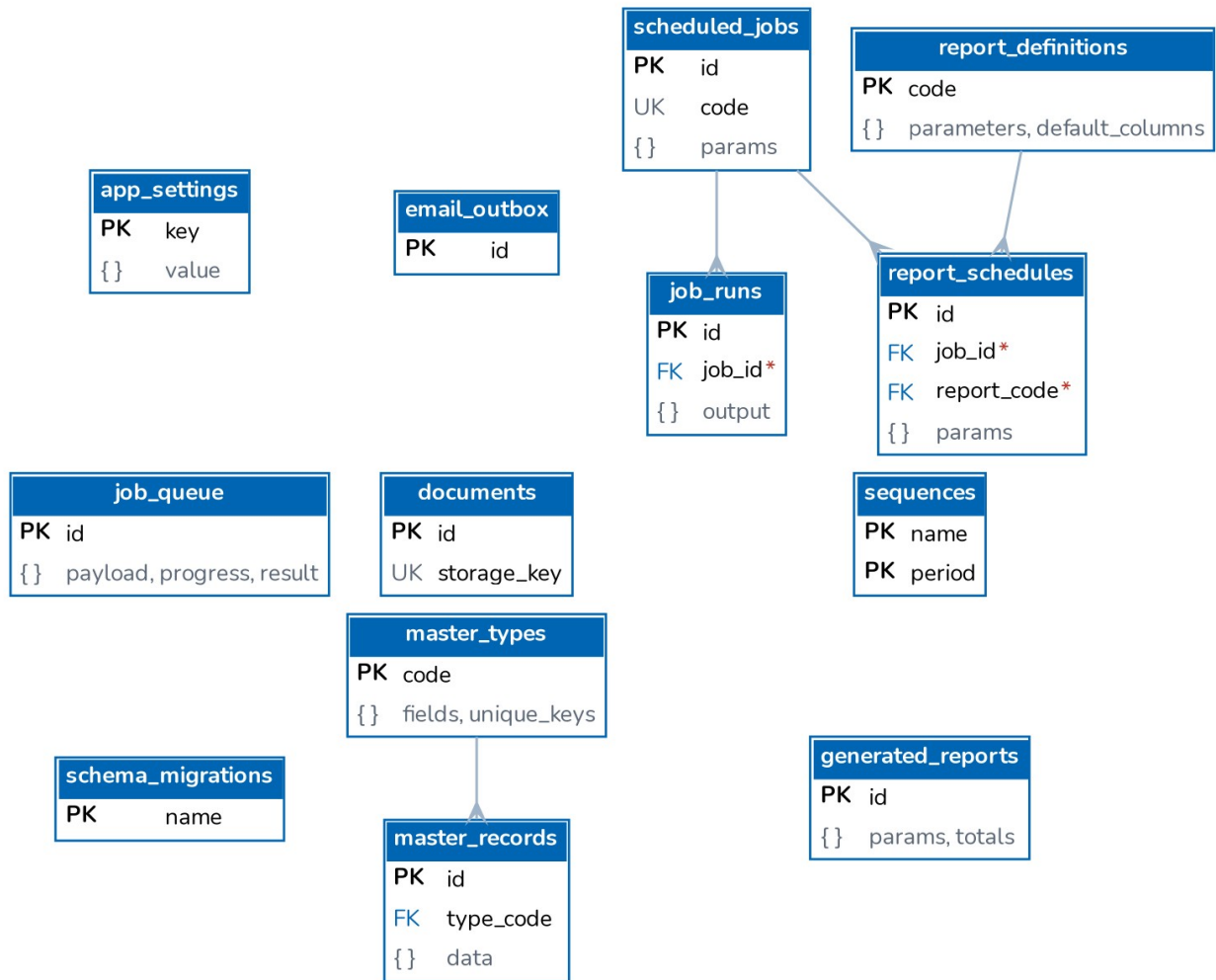


Figure 10. Configuration and platform services tables

4.10 Reference masters and product configurator

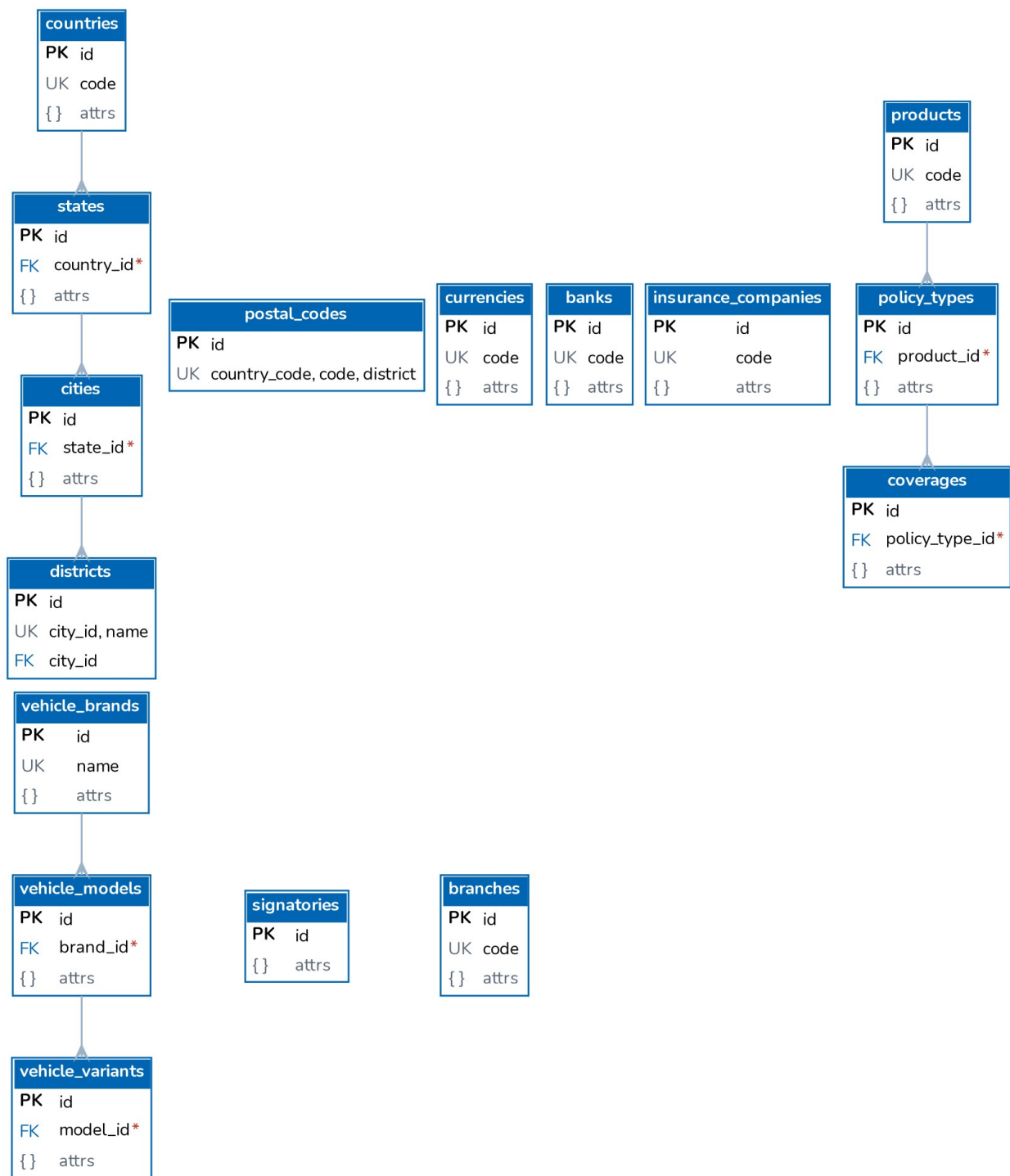


Figure 11. Reference masters

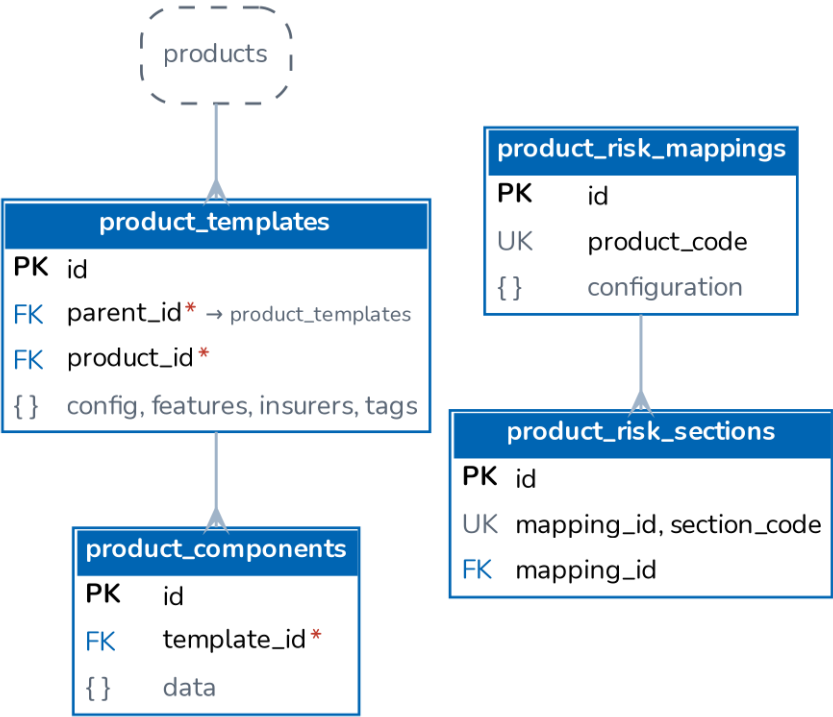


Figure 12. Product configurator

5 Migrations

Aspect	Implementation (<code>backend/src/db/migrate.js</code>)
Format	Plain SQL files <code>NNNN_name.sql</code> in <code>backend/src/db/migrations</code> , applied in file-name order. Numbers are grouped by area: 0001-0004 core, masters, operations, finance; 0010-0021 sales, claims, renewals; 0030-0034 ledger, receipts, disbursements, commission, petty cash; 0040-0044 configuration, product configurator, remittance, reinsurance, incentive; 0050 reports; 0060 security; 0070-0097 later changes (policy payments, chart of accounts, direct bill, security hardening, least-privilege reads, notification audience, principal insurer); 0110 branding.
Tracking	Table <code>schema_migrations(name, applied_at)</code> ; a file is applied once.
Execution	On every API start (<code>server.js</code>), before the seed: each pending file runs in its own transaction together with its <code>schema_migrations</code> row; on error the transaction is rolled back and the API does not start. <code>npm run migrate</code> runs the same code; <code>npm run db:reset</code> drops and recreates the schema (development only).
Idempotency	Files use <code>CREATE TABLE IF NOT EXISTS</code> , <code>ADD COLUMN IF NOT EXISTS</code> , <code>CREATE INDEX IF NOT EXISTS</code> and guarded data updates, so a file partially applied by hand can be re-run.
Readiness	<code>GET /api/health</code> answers 503 until no migration is pending, so a load balancer does not route to an instance with an older schema.
Direction	Forward only (no down migrations). The go-live checklist states that migrations only add; rollback of data means restoring a snapshot (document 09).

Recommended / to be confirmed by the business and DevOps: With several instances starting together, two instances may try to apply the same pending file; the second fails on the unique `schema_migrations` row or on the DDL and restarts. Deploy with one instance first (as the checklist says) or add a PostgreSQL advisory lock around `migrate()` (small code change). Keep destructive changes (column drops, type changes) in a separate release after the code that no longer uses them is live.

6 Seed data

`backend/src/db/seed.js` runs on every start after the migrations and with `npm run seed`. Every step is idempotent: rows are inserted by natural key or fixed id (`ON CONFLICT DO NOTHING / DO UPDATE` on descriptive columns), so administrator edits are kept.

1. In code: the 9 roles, 50 permissions (read / write for 25 modules), the role grants, and the administrator **BrokerVerse** (password from `ADMIN_PASSWORD`; without it a random password is generated, logged once and must be changed at first sign-in).
2. `seeds/settings.json` (configuration keys, inserted only if missing) and `seeds/jobs.json` (the 9 scheduled jobs).
3. Encryption of any two-factor secret still stored in clear (upgrade path).
4. The SQL files `seeds/*.sql` (**reference data**, always) and `seeds/sample/*.sql` (**sample data**, only when `SEED_SAMPLE_DATA` is on), interleaved by file name so a sample file runs after the reference file it depends on.

Class	Content
Reference	Settings; roles and permissions; Philippine geography, districts and postal codes; currencies, banks, insurers, products and policy types; vehicle makes, models and variants; BI / PD / PA covers; head office branch; master screen registry and reference master records; chart of accounts (128 accounts) and main / sub account masters; product templates with the motor and CTPL tariff; report catalogue (19 reports); direct-bill and security configuration.
Sample	Fictional leads, clients, quotations, policies, endorsements, claims, renewals, receivables, receipts, collections, disbursements, petty cash, commissions and referrers, remittances, reinsurance, incentives, journal vouchers, demo branches, signatories and master records, and six sample users.

`SEED_SAMPLE_DATA` accepts true / false (1 / 0, yes / no, on / off); unset, it is on in development and test and off with `NODE_ENV=production`. A database seeded with sample data is cleaned with `npm run purge:sample` (document 10).

7 Time zone, date and amount handling

Topic	Design
Storage	Instants in <code>timestamp_tz</code> (stored in UTC); calendar dates in <code>date</code> . The server and database run in UTC (local server <code>Etc/UTC</code> ; the container images default to UTC).
Business date	<code>lib/dates.js</code> : "today" is the calendar date in <code>general.timezone</code> (seeded <code>Asia/Manila</code>), not the server's UTC date; <code>businessDate()</code> , <code>calendarPeriod()</code> and the report engine use the configured zone, so between 00:00 and 08:00 Manila time the system does not use yesterday's date.
Display	Front end formats dates with <code>general.date_format (DD/MM/YYYY)</code> through <code>utility/dateFormat.js</code> ; amounts with the configured currency and locale (<code>useFormatCurrency</code> , <code>formatMoney()</code> on the server).
Money	<code>numeric</code> in the database, rounding by <code>round2()</code> (half away from zero with binary floating-point correction); taxes (VAT 12 %, DST 12.5 %, LGT 0.75 %) and commission rates come from settings.

Gap: A few SQL statements use the database date instead of the business date: the `policy-expiry` job (`expiry_date < current_date`), the `quote-expiry` job (`created_at < now() - interval`), the renewal pipeline (`current_date + days`) and `next_number()` (year). With the database in UTC, these run against the UTC date, which differs from the Manila date between 00:00 and 08:00 Manila time. Combined with the scheduler running in UTC (document 01), the daily jobs currently run at 08:15-16:00 Manila time, when the UTC and Manila dates agree, so the effect today is limited to the year boundary of document numbers; it becomes visible if the scheduler time zone is fixed without also fixing these statements. Recommended: set the database role's `timezone` to `Asia/Manila` (`ALTER ROLE ... SET timezone`) or pass the business date as a parameter.

Note: Amounts are converted from `numeric` to JavaScript double-precision numbers. This is exact for amounts of up to about 90 trillion with two decimals, far above broking volumes, provided every arithmetic result is rounded with `round2()`, which the ledger and document code does.

8 Related documents

This document is one of eleven that together form the BrokerVerse Solution Architecture. They are maintained and issued together (source and build: [docs/architecture/README.md](#)).

No.	Document	File
01	Solution Component Diagram and Application Architecture	01_BrokerVerse_Solution_Component_Diagram_Application_Architecture.pdf
02	Database Design (this document)	02_BrokerVerse_Database_Design.pdf
03	Database Inventory	03_BrokerVerse_Database_Inventory.pdf
04	Technology Stack	04_BrokerVerse_Technology_Stack.pdf
05	Shared Service Components	05_BrokerVerse_Shared_Service_Components.pdf
06	Capacity and Performance	06_BrokerVerse_Capacity_and_Performance.pdf
07	High Availability and Resiliency	07_BrokerVerse_High_Availability_and_Resiliency.pdf
08	RTO and RPO	08_BrokerVerse_RTO_and_RPO.pdf
09	Backup and Recovery	09_BrokerVerse_Backup_and_Recovery.pdf
10	Data Archival, Housekeeping and Restoration	10_BrokerVerse_Data_Archival_Housekeeping_and_Restoration.pdf
11	Monitoring	11_BrokerVerse_Monitoring.pdf

9 Glossary

Terms and abbreviations used in this document.

Term	Meaning
Advisory lock	PostgreSQL application-level lock (<code>pg_try_advisory_lock</code>) used by the scheduler so a job runs on one API instance at a time.
API	Application programming interface; here the BrokerVerse REST API under <code>/api</code> served by the Node.js backend.
Audit log / audit trail	Table <code>audit_log</code> : who changed which record, when, from which IP, with the before and after values.
Bordereau	Periodic list of ceded premiums or claims sent to a reinsurer.
Cession	The share of a policy's risk and premium passed to a reinsurer under a treaty or facultatively.
CTPL	Compulsory Third Party Liability motor insurance (Philippines).
Debit note (DN)	Commission debit note raised by the broker to an insurer for direct-billed policies.
Direct bill	Billing mode in which the insurer collects the premium and the broker bills the insurer for its commission.
DST	Documentary Stamp Tax (Philippines), part of the gross premium.
Endorsement	Change to an issued policy (details, cover, premium or cancellation).
FK	Foreign key: column that references the primary key of another table.
GIN	Generalised inverted index (PostgreSQL), used for JSONB searches.
JSONB	PostgreSQL binary JSON column type, used for flexible document parts of records.
LGT	Local Government Tax (Philippines), part of the gross premium.
Migration	Numbered SQL file in <code>backend/src/db/migrations</code> applied once, in order, on API start.
OR	Official receipt issued for premium received.
PK	Primary key: column(s) that uniquely identify a row.
RDS	Amazon Relational Database Service, the managed PostgreSQL target.
Receivable / bill	Premium bill (INV-) raised when a policy, endorsement or renewal is issued.
Remittance	Payment of collected premium, net of commission, from the broker to the insurer.
RPO	Recovery point objective: the maximum acceptable data loss, measured in time.
RTO	Recovery time objective: the maximum acceptable time to restore service.
Seed	Idempotent data load run on every API start: reference data always, sample data only when <code>SEED_SAMPLE_DATA</code> is on.
SLA	Service level agreement / target time, e.g. for remittance approvals.
Treaty	Reinsurance contract under which risks are ceded automatically within agreed limits.
UAT	User acceptance testing environment.
VAT	Value-added tax (12 % in the Philippines).
zod	TypeScript-first schema validation library used to validate request bodies and parameters.